

Proyecto Final: API de Gestión de Rutinas de Entrenamiento

Desarrollar el **backend completo** de una aplicación web para gestionar rutinas de entrenamiento. El frontend ya ha sido proporcionado en esta misma tarea y se conecta mediante una API RESTful. La aplicación permitirá a los usuarios registrarse, iniciar sesión, consultar rutinas públicas y crear sus propias rutinas

Estructura general del backend

```
TrainUp/
|
|— main.py                                # Punto de entrada de la aplicación
FastAPI
|— db.py
|— models/                                # Modelos ORM (estructura de la base de
datos)
|   |— usuario.py                        # Modelo Usuario
|   |— rutina.py                        # Modelo Rutina (con relación
muchos-a-muchos con Ejercicio)
|   |— ejercicio.py                    # Modelo Ejercicio
|   |— __init__.py                     # Importación de todos los modelos
|
|— schemas/                               # Esquemas Pydantic para validación de
entrada/salida
|   |— usuario.py                      # Esquemas para Usuario
|   |— rutina.py                      # Esquemas para Rutina
|   |— ejercicio.py                   # Esquemas para Ejercicio
|   |— __init__.py                   # Importación de todos los esquemas
|
|— repositories/                         # Capa de acceso a datos (repositorios)
|   |— base.py                        # Repositorio base genérico
|   |— usuarios.py                   # Repositorio de usuarios
|   |— rutinas.py                    # Repositorio de rutinas
|   |— ejercicios.py                 # Repositorio de ejercicios
|   |— __init__.py                   # Importación de todos los repositorios
|
|— routers/                              # Routers de FastAPI
|   |— usuarios.py                   # Endpoints relacionados con usuarios
|   |— rutinas.py                    # Endpoints de rutinas
|   |— ejercicios.py                 # Endpoints de ejercicios
|   |— __init__.py                   # Importación de routers
```

El proyecto está organizado en módulos que separan responsabilidades. A continuación se describe cada uno y lo que se espera que implementes en ellos:

1. main.py - Punto de entrada

- Configura FastAPI.
- Habilita CORS para que el frontend pueda comunicarse con el backend.
- Carga los modelos y crea las tablas.
- Incluye los routers correspondientes a usuarios, rutinas y ejercicios.

Debe ser algo similar a esto:

```
# main.py

from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from db import Base, engine, SessionLocal

# Importamos los modelos explícitamente para que se creen las tablas
from models import usuario, rutina, ejercicio

# Importamos los routers
from routers import usuarios, rutinas, ejercicios

from db import SessionLocal
from models.usuario import Usuario
from models.rutina import Rutina
from models.ejercicio import Ejercicio

# Creamos la instancia principal de la app FastAPI
app = FastAPI(title="API TrainUp 2.0")

# Configuramos CORS para permitir que el frontend acceda a la API
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://127.0.0.1:5500"], # Durante desarrollo,
    permitimos cualquier origen
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
# Creamos las tablas automáticamente si no existen
Base.metadata.create_all(bind=engine)

# Incluimos los routers de cada recurso
app.include_router(usuarios.router, prefix="/api/usuarios")
app.include_router(rutinas.router, prefix="/api/rutinas")
app.include_router(ejercicios.router, prefix="/api/ejercicios")

# Ruta simple para probar que la API funciona
@app.get("/")
def inicio():
    return {"mensaje": "Bienvenido a TrainUp API"}
```

2. db.py - Conexión con la base de datos

- Define el motor de base de datos y la clase SessionLocal.
- Define la base Base de SQLAlchemy sobre la que se construirán los modelos.

Debe ser algo similar a esto:

```
# db.py

from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# URL de conexión a la base de datos SQLite (archivo local)
DATABASE_URL = "sqlite:///./trainup.db"

# Creamos el motor de conexión
engine = create_engine(
    DATABASE_URL, connect_args={"check_same_thread": False} # Necesario para
    SQLite
```

```
)

# Creamos la clase SessionLocal para abrir sesiones con la base de datos
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# Declarative Base es la clase base para todos los modelos
Base = declarative_base()
```

3. models/ – Modelos ORM

En este directorio se encuentran los modelos de datos utilizados por SQLAlchemy para definir la estructura de las tablas de la base de datos. Cada clase representa una tabla y cada atributo un campo en dicha tabla. También se incluyen las relaciones entre entidades.

models/usuario.py

Contiene la clase Usuario, que representa a un usuario registrado en la plataforma. Este modelo incluye los siguientes atributos:

- id: Integer, clave primaria, autoincremental.
- nombre: String(100), obligatorio.
- email: String(100), único y obligatorio.
- telefono: String(20), obligatorio.
- contraseña: String(100), obligatorio.

Relación:

- **Uno a muchos** con Rutina: un usuario puede haber creado varias rutinas. Esto se modela mediante el atributo rutinas, con una relación relationship("Rutina", back_populates="usuario").

models/ejercicio.py

Contiene la clase Ejercicio, que representa un ejercicio de entrenamiento. Sus atributos son:

- id: Integer, clave primaria, autoincremental.
- nombre: String(100), obligatorio.
- grupoMuscular: String(50), obligatorio.
- series: Integer, obligatorio.
- repeticiones: Integer, obligatorio.

Relación:

- **Muchos a muchos** con Rutina: un mismo ejercicio puede estar presente en varias rutinas, y cada rutina puede incluir múltiples ejercicios. Esto se define mediante el atributo rutinas, que se conecta con la tabla intermedia rutina_ejercicio.

models/rutina.py

Contiene la clase Rutina, que representa una rutina de entrenamiento personalizada. Sus atributos son:

- id: Integer, clave primaria, autoincremental.
- nombre: String(100), obligatorio.
- descripcion: String(300), opcional.
- nivel: String(30), obligatorio.
- usuarioid: Integer, clave foránea que enlaza con Usuario.id.

Relaciones:

- **Uno a muchos** con Usuario: cada rutina pertenece a un único usuario. A través del atributo usuario = relationship("Usuario", back_populates="rutinas").
- **Muchos a muchos** con Ejercicio: definido mediante la relación ejercicios = relationship("Ejercicio", secondary=rutina_ejercicio, back_populates="rutinas", lazy="joined"). **En este caso, será importante poner el lazy="joined".**

Relación muchos a muchos entre Rutina y Ejercicio

Esta relación está definida mediante una tabla intermedia llamada `rutina_ejercicio`, declarada como un `Table` en `models/ejercicio.py`. Esta tabla no tiene un modelo propio, pero actúa como enlace entre ambas entidades. Sus columnas son:

- `rutina_id`: Integer, clave foránea a `rutinas.id`, clave primaria compuesta.
- `ejercicio_id`: Integer, clave foránea a `ejercicios.id`, clave primaria compuesta.

Esta estructura permite reutilizar ejercicios ya existentes en múltiples rutinas, y que una rutina se construya a partir de un conjunto de ejercicios predefinidos.

4. schemas/ – Esquemas de validación con Pydantic

Los esquemas Pydantic definen cómo se validan los datos de entrada (al crear o modificar entidades) y cómo se serializan los datos de salida (al devolver respuestas desde la API). Estos esquemas permiten una validación robusta y estructurada entre la lógica de negocio y la capa de presentación (frontend o cliente).

schemas/usuario.py

- **UsuarioCrear**: esquema para el registro de un nuevo usuario.
 - `nombre (str)`: nombre completo del usuario.
 - `email (str)`: dirección de correo electrónico.
 - `telefono (str)`: número de teléfono.
 - `contrasena (str)`: contraseña del usuario (en texto plano; se recomienda cifrarla, pero ya veremos como más adelante).
- **UsuarioLogin**: esquema para realizar login de usuario.
 - `email (str)`: correo electrónico.
 - `contrasena (str)`: contraseña.
- **UsuarioRespuesta**: esquema de salida que expone los datos del usuario (sin contraseña).
 - `id (int)`: identificador único.
 - `nombre (str)`
 - `email (str)`

- telefono (str)
- Usa orm_mode = True para permitir la conversión desde instancias ORM.

schemas/ejercicio.py

- **EjercicioCrear:**

- nombre (str)
- grupoMuscular (str)
- series (int)
- repeticiones (int)

- **EjercicioRespuesta:** esquema que se devuelve al consultar ejercicios.

- id (int)
- nombre (str)
- grupoMuscular (str)
- series (int)
- repeticiones (int)
- También usaría orm_mode = True

schemas/rutina.py

- **RutinaCrear:**

- nombre (str)
- descripcion (Optional[str]): descripción general de la rutina.
- nivel (str): dificultad de la rutina (e.g., “principiante”, “intermedio”, “avanzado”).
- usuariold (int): ID del usuario que crea la rutina.
- ejerciciosIds (List[int]): lista de IDs de ejercicios ya existentes que se asociarán a esta rutina.

- **RutinaRespuesta:**

- id (int)
- nombre (str)
- descripcion (Optional[str])
- nivel (str)
- usuarioid (int)
- ejercicios (List[EjercicioRespuesta]): lista completa de objetos ejercicio relacionados con la rutina.
- También usaría orm_mode = True

4. repositories/ – Repositorios de acceso a datos

Los repositorios encapsulan la lógica de acceso a la base de datos. Cada uno debe ofrecer funciones específicas para gestionar las operaciones CRUD y búsquedas necesarias en la aplicación.

repositories/base.py

Contiene una clase base RepositorioBase genérica que ofrece los métodos comunes para todos los repositorios:

- crear(sesion, datos): Crea un nuevo ejercicio con sus datos básicos (nombre, grupo muscular, series, repeticiones).
- obtener_todos(): recupera todas las instancias del modelo.
- obtener_por_id(id): busca una instancia por su identificador.
- borrar(id): elimina una instancia de la base de datos.
- actualizar(id, nuevos_datos): actualiza los datos de una instancia.

Todos los demás repositorios heredan de esta clase y añaden sus métodos específicos.

repositories/usuarios.py

Repositorio centrado en la gestión de usuarios. Debe implementar:

- crear(sesion, datos): Crea un nuevo usuario con los datos recibidos (nombre, email, teléfono, contraseña).
- obtener_por_email(sesion, email): Busca y devuelve un usuario a partir del correo electrónico. Será útil en el login para autenticar usuarios registrados.

Modelo asociado: Usuario

repositories/ejercicios.py

Repositorio para manejar los ejercicios del sistema. El alumno debe implementar:

- obtener_por_rutina(sesion, rutina_id): Recupera los ejercicios que están asociados a una rutina concreta.
- obtener_todos(sesion): Devuelve todos los ejercicios disponibles.

Modelo asociado: Ejercicio

repositories/rutinas.py

Su objetivo es manejar tanto las rutinas como su relación con los ejercicios (relación muchos a muchos).

Debe implementar:

- obtener_por_usuario(sesion, usuario_id): Devuelve todas las rutinas creadas por un usuario concreto, también con sus ejercicios cargados.
- buscar_por_nombre(sesion, nombre): Permite buscar rutinas que contengan un nombre parcial, útil para la barra de búsqueda del frontend.

Modelo asociado: Rutina, con relación many-to-many con Ejercicio usando una tabla intermedia rutina_ejercicio

5. routers/ – Rutas de la API (FastAPI)

Los routers agrupan y organizan los endpoints que ofrece la API. Cada recurso tiene su propio archivo y define las rutas necesarias para interactuar con los datos. **Será crucial utilizar los esquemas aquí**

routers/usuarios.py

Este router gestiona todo lo relacionado con los usuarios de la aplicación:

POST /api/usuarios: Crea un nuevo usuario.

Espera un JSON con los siguientes campos:

```
{
  "nombre": "string",
  "email": "string",
  "telefono": "string",
  "contrasena": "string"
}
```

POST /api/usuarios/login: Inicia sesión.

Verifica si existe un usuario con el email y la contraseña recibidos.

Entrada esperada:

```
{
  "email": "string",
  "contrasena": "string"
}
```

- **GET /api/usuarios/{id}/rutinas:** Devuelve todas las rutinas creadas por el usuario con ID indicado.

Es importante comprobar que el inicio de sesión valida adecuadamente las credenciales y devuelve los datos del usuario si son correctos.

routers/rutinas.py

Este router gestiona las operaciones sobre rutinas:

- **GET /api/rutinas:** Lista todas las rutinas.
- **GET /api/rutinas/{id}:** Devuelve los detalles de una rutina específica, la especificada por id, incluyendo su lista de ejercicios asociados.

POST /api/rutinas: Crea una nueva rutina y la vincula a una lista de ejercicios ya existentes.

Entrada esperada:

```
{
  "nombre": "string",
  "descripcion": "string",
  "nivel": "string",
  "usuarioId": 1,
  "ejerciciosIds": [1, 3, 5]
}
```

Es fundamental implementar correctamente la relación muchos-a-muchos con ejercicios al crear una rutina, asegurándose de que los ejercicios existen y el usuario asociado también.

routers/ejercicios.py

Este router gestiona todo lo relacionado con los ejercicios de la aplicación. Permite consultar y registrar ejercicios que luego pueden asociarse a rutinas.

GET /api/ejercicios:

Devuelve la lista completa de ejercicios registrados en la base de datos.

Esta ruta es útil para rellenar el selector de ejercicios al crear una nueva rutina en el frontend.

POST /api/ejercicios:

Permite crear un nuevo ejercicio.

Entrada esperada:

```
{
  "nombre": "string",
  "grupoMuscular": "string",
  "series": 3,
  "repeticiones": 12
}
```

Cada ejercicio puede ser reutilizado en múltiples rutinas gracias a la relación muchos-a-muchos con el modelo Rutina. Es importante comprobar que los datos introducidos (número de series, repeticiones, etc.) sean válidos antes de insertarlos.

